Contents

Chapter 1

Acknowledgements

This project would not have been possible without the support, guidance, and collaboration of many individuals, to whom we are deeply grateful.

First and foremost, we would like to express our sincere gratitude to our project supervisor, **Dr. Tawfikur Rahman**, for his continuous guidance, encouragement, and insightful feedback throughout every stage of the project. His expertise played a pivotal role in shaping both the technical and practical dimensions of our work.

We are especially thankful to one another for the dedication, teamwork, and shared commitment that each team member brought to the table. This was truly a collaborative effort, with everyone contributing significantly to the project's development and completion.

We also gratefully acknowledge the valuable support and advice provided by our seniors. Their experience and suggestions helped us overcome several challenges during the course of the project.

Lastly, we extend our appreciation to the faculty members and the institution for providing the necessary resources, infrastructure, and a supportive environment that enabled the successful completion of this work.

Chapter 2

Abstract

The rapid advancement of the Internet of Things (IoT) has enabled the development of low-cost, portable, and efficient vision-based systems. This project presents the design and implementation of a fruit detection system using an ESP32-CAM module integrated with a laptop-based inference pipeline. The ESP32-CAM captures images and transmits them over a local wireless network to a laptop, where a pre-trained YOLOv8m object detection model is deployed. The model, trained on a custom dataset using Google Colab with GPU acceleration, identifies fruits in real time and overlays bounding boxes with class labels.

The system architecture combines lightweight embedded hardware with high-performance deep learning models, thereby balancing cost-efficiency and detection accuracy. The ESP32-CAM, powered by a portable power bank, serves as a continuous image source while maintaining mobility. A Python-based script running on the laptop processes incoming frames, performs inference, and displays live video with detection overlays.

Experimental results demonstrate that the system is capable of detecting fruits with high accuracy while maintaining real-time visualization. Despite challenges related to ESP32 memory limitations, network stability, and Python inference delays, the project validates the feasibility of combining IoT devices with modern computer vision frameworks. This work highlights the potential for low-cost smart agricultural applications, automated monitoring, and future extensions involving edge deployment, cloud integration, and real-time alert systems.

Chapter 3

Introduction

3.1 Background and Motivation

The integration of the Internet of Things (IoT) with computer vision has enabled cost-effective solutions for real-time monitoring and automation. Fruit detection, in particular, plays a vital role in agriculture, food quality control, and supply chain management. Traditional inspection methods are labor-intensive and prone to errors, while advanced automated systems often require expensive hardware. The ESP32-CAM offers a low-cost imaging solution but lacks the computational capacity to run modern deep learning models. This project is motivated by the need to bridge this gap through a hybrid system that combines IoT-enabled image capture with external deep learning inference.

3.2 Problem Definition

Manual fruit identification and classification can be time-consuming, inconsistent, and prone to human error. Existing automated systems often rely on costly, high-performance hardware, making them inaccessible for small-scale or experimental use. The ESP32-CAM is a low-cost IoT device with a built-in camera, but its limited processing capability prevents direct deployment of modern deep learning models such as YOLO. This creates a gap between the affordability of IoT hardware and the computational requirements of state-of-the-art object detection frameworks. The problem addressed in this project is how to design a low-cost, IoT-enabled fruit detection system that leverages the ESP32-CAM for image acquisition while offloading heavy inference tasks to an external computing device.

3.3 Objectives of the Project

The objectives of the project are as follows:

- To implement a fruit detection system using ESP32-CAM and a YOLOv8m model.
- To establish a wireless communication link for image transfer between ESP32-CAM and laptop.
- To train a custom YOLOv8m model using Google Colab GPU resources.
- To develop a Python-based application for inference, bounding box visualization, and live video display.
- To evaluate system performance in terms of detection accuracy, latency, and frame rate.

3.4 Scope of the Project

This project focuses on building a prototype-level IoT-based fruit detection system rather than a fully deployed commercial solution. The scope includes:

- **Hardware:** ESP32-CAM module, ESP32 development board, laptop, and power bank.
- **Software:** Google Colab for training, Python scripts for inference, and YOLOv8m as the detection model.
- **Functionality:** Real-time fruit detection with bounding boxes and labels displayed on live video.
- **Limitations:** Inference is performed on the laptop instead of the ESP32 due to hardware constraints. Performance may vary depending on network stability and processing power of the laptop.
- **Future Extensions:** Edge deployment with optimized models, cloud-based scaling, and integration of real-time notifications or decision-making systems.

3.5 Report Structure

The report is structured as follows:

- Chapter 4 reviews related literature on IoT, ESP32-CAM, YOLO models, and comparative studies.
- Chapter 5 presents the system overview, architecture, and tools used.
- Chapter 6 explains the methodology, including dataset preparation, training, and deployment.
- Chapter 7 details the implementation of ESP32-CAM, Python scripts, and live video processing.
- Chapter 8 provides experimental results and analysis.
- Chapter 9 demonstrates system outputs and performance evaluation.
- Chapter 10 discusses challenges and limitations.
- Chapter 11 outlines future scope and recommendations.
- Chapters 12–14 present the conclusion, references, and appendices.

Chapter 4

Literature Review

4.1 Applications of IoT in Image Processing

The integration of IoT with image processing has enabled real-time monitoring, analysis, and decision-making across diverse domains. In agriculture, IoT-based systems monitor crops, detect pests, and classify fruits and vegetables. In healthcare, medical imaging combined with IoT facilitates remote diagnostics and patient monitoring. Security and surveillance systems utilize IoT-enabled cameras for motion detection, facial recognition, and anomaly detection. These applications leverage low-cost sensors, wireless communication, and cloud or edge computing, allowing large-scale, distributed image acquisition and processing with minimal human intervention.

4.2 ESP32-CAM in Vision-Based Systems

The ESP32-CAM module is a compact, low-cost embedded device featuring an integrated camera and Wi-Fi connectivity. It is widely used in vision-based IoT systems such as home surveillance, face detection, and remote monitoring. Despite its advantages in portability and affordability, the ESP32-CAM is constrained by limited onboard memory and processing power, which restricts the deployment of resource-intensive deep learning models. To overcome these limitations, ESP32-CAM-based systems often stream images to external computing devices for inference, making it a practical choice for cost-effective prototyping and IoT-based imaging solutions.

Chapter 5

System Overview

5.1 Hardware Components

The system uses the following hardware:

- **ESP32-CAM Module:** Captures and streams images over Wi-Fi.
- **ESP32 Development Board:** Provides programming and debugging interface.
- **Laptop:** Runs inference, visualization, and performance evaluation.
- **USB Type-B Cable:** Connects the ESP32 development board to the laptop for programming.
- **Power Bank:** Supplies continuous 5v to the ESP32-CAM for mobile operation.

5.2 Software Tools and Libraries

The software stack includes Google Colab for training the YOLOv8m model using GPU resources, and Python 3.x for implementing the inference and visualization pipeline. OpenCV is used for image processing and displaying live video streams with detection overlays. The Ultralytics YOLOv8 library provides the object detection framework. Additionally, the ESP32-CAM firmware is developed using Arduino IDE , and a lightweight web server facilitates image streaming from the ESP32-CAM to the laptop.

5.3 System Design Diagram

A graphical representation of the full system is provided below. The diagram includes the following components with arrows indicating data flow:

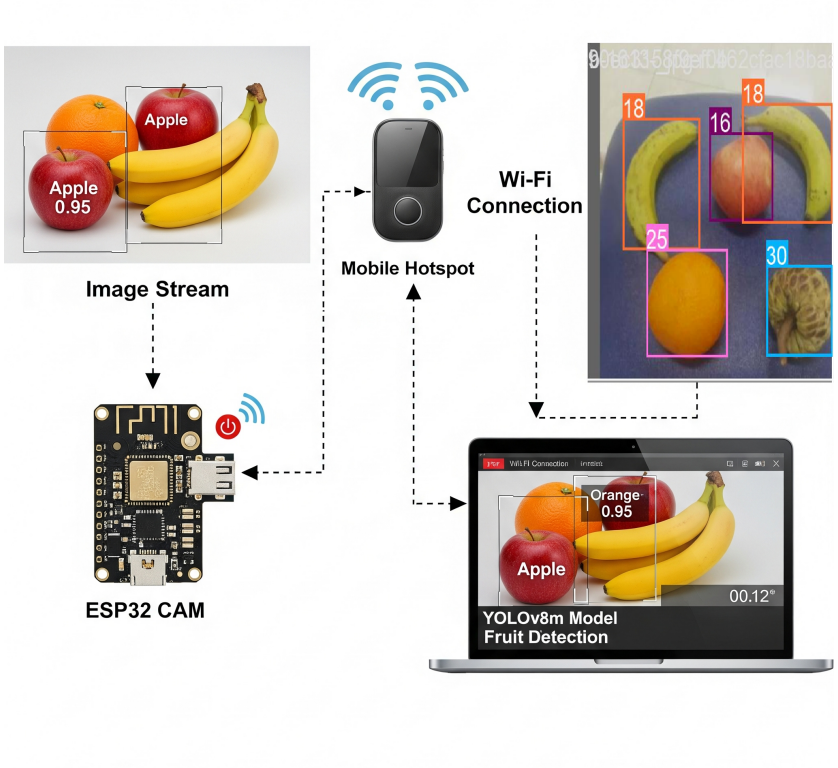

Figure 5.1: System Architecture Diagram

Chapter 6

Methodology

6.1 Data Collection and Dataset Preparation

The dataset used for fruit detection was collected from Roboflow's open-source repository, specifically the *Fruits and Veg 2* dataset [2]. This dataset contains a diverse set of fruit and vegetable images with annotated bounding boxes and class labels suitable for supervised learning.

Additional preprocessing steps, such as resizing, normalization, and image augmentation (including rotation, flipping, and brightness adjustment), were applied to increase model robustness against variations in lighting, scale, and orientation. The dataset was then divided into training, validation, and test subsets to facilitate reliable evaluation and prevent overfitting.

6.2 Model Selection and Training using Google Colab

YOLOv8m was selected for its high detection accuracy and real-time inference capabilities. The model was trained on Google Colab using a T4 GPU to accelerate computations. Training involved optimizing hyperparameters such as learning rate, batch size, and number of epochs to achieve high precision and recall. During training, metrics such as loss curves, mean average precision, and confusion matrices were monitored to assess convergence and model performance. The trained model weights were then exported for deployment on the laptop.

6.3 Network Setup and Communication Design

A mobile hotspot served as the primary wireless network, connecting both the ESP32-CAM and the laptop. The ESP32-CAM continuously captured frames and transmitted them over the network to a lightweight HTTP server running on the laptop. This design ensures low-latency communication and real-time data availability. Network parameters, such as IP addressing, port configuration, and streaming intervals, were optimized to minimize packet loss and maintain smooth video streaming.

6.4 Deployment Pipeline

- **Image Capture:** The ESP32-CAM continuously captures frames of the scene containing fruits.
- **Image Streaming:** Captured frames are transmitted over a mobile hotspot to the laptop via a lightweight HTTP server.
- **Inference:** The laptop runs a Python script that inputs each frame into the trained YOLOv8m model.
- **Detection Overlay:** Bounding boxes and class labels are drawn on the detected fruits for visualization.
- **Live Display:** Annotated frames are displayed as a continuous video stream on the laptop.
- **Performance Logging:** Metrics such as frame rate, latency, and detection accuracy are recorded for evaluation.
- **Optimization:** Streaming intervals and network parameters are tuned to ensure low latency and real-time performance.

Chapter 7

Implementation Details

7.1 ESP32-CAM Setup and Firmware

The ESP32-CAM module is connected to the ESP32 development board via a USB Type-B interface for programming. A power bank supplies continuous power, enabling mobile operation. The firmware is developed using Arduino IDE with the `esp32cam` library, allowing the module to capture and stream images over a Wi-Fi network.

Key points of the ESP32-CAM setup include:

- Camera initialized with configurable resolutions: low, medium, and high.
- Wi-Fi credentials set to connect the ESP32-CAM to a mobile hotspot.
- HTTP server endpoints (`/cam-lo.jpg`, `/cam-mid.jpg`, `/cam-hi.jpg`) serve frames to client devices.
- Continuous handling of HTTP requests with error detection for failed captures.

The main loop of the firmware continuously waits for client requests. When a request is received, the ESP32-CAM captures a frame, validates it, and sends it as a JPEG response. Resolution handlers allow flexibility in balancing image quality and bandwidth. The system ensures reliability even under network fluctuations or camera capture failures.

7.2 Python-Based Inference and Live Video Visualization

The Python script on the laptop handles receiving images, performing inference, and displaying live annotated video. The workflow is as follows:

- **Frame Acquisition:** HTTP requests fetch JPEG frames from the ESP32-CAM. Frames are decoded into NumPy arrays for processing.
- **YOLOv8m Inference:** The trained YOLOv8m model (`best.pt`) predicts bounding boxes and class labels for detected fruits.
- **Annotation Overlay:** Bounding boxes, fruit names, and confidence scores are drawn on each frame using OpenCV.
- **Live Display:** Annotated frames are shown in a continuous video window. The display updates in real-time and can be terminated with a keypress (`q`).
- **Error Handling and Optimization:** Failed requests, corrupted frames, or decoding errors are skipped without interrupting the loop. Small delays help maintain smooth frame rates.

This integrated pipeline ensures that the system can capture images from the ESP32-CAM, process them with the YOLOv8m model, and provide a real-time visualization of detected fruits with high responsiveness and accuracy.

Chapter 8

Experimental Results and Analysis

8.1 Training Metrics and Evaluation Results

The YOLOv8m model was trained on the prepared fruit dataset using Google Colab with a T4 GPU. Training metrics, including loss curves and mean average precision (mAP), were monitored to assess convergence and performance. The Precision-Recall (PR) curve (`Figure 8.1:Precision-Recall Curve`) demonstrates the trade-off between precision and recall across all classes, highlighting the model's ability to balance false positives and false negatives.

The confusion matrix (`Figure 8.2:confusion-matrix`) further evaluates class-wise prediction performance, revealing which fruit classes were most accurately detected and which were occasionally misclassified. Overall, the metrics indicate that the trained model achieves high detection accuracy while maintaining reliable performance across diverse fruit types and imaging conditions.

8.2 Analysis of Key Output Images

Annotated images (`Figure 8.3:Labels`) demonstrate correct detection of multiple fruits with bounding boxes and confidence scores. The summary results (`Figure 8.4:Summary Results`) provide an overview of detection outputs, confirming that the ESP32-CAM and YOLOv8m pipeline works effectively for real-time fruit detection.

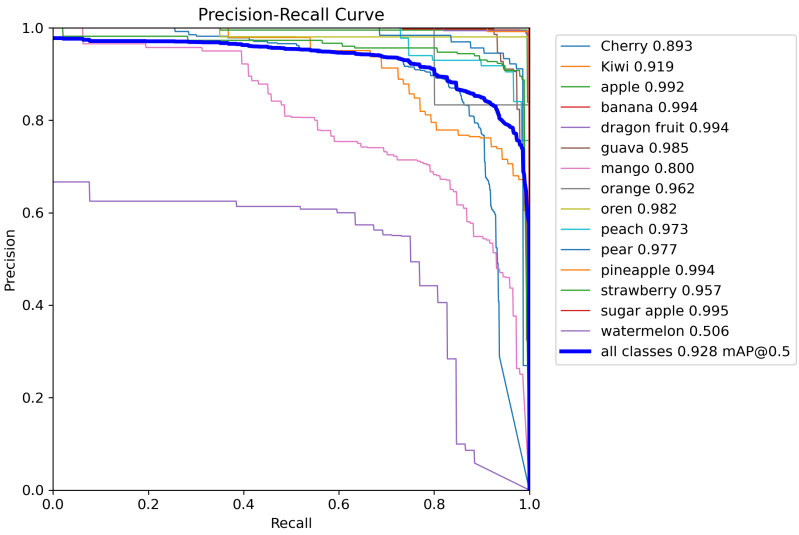

Figure 8.1: Precision-Recall Curve

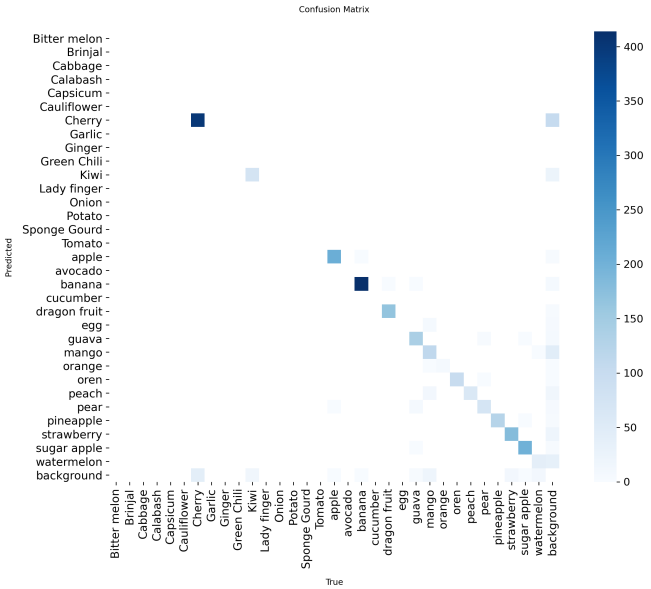

Figure 8.2: Confusion Matrix

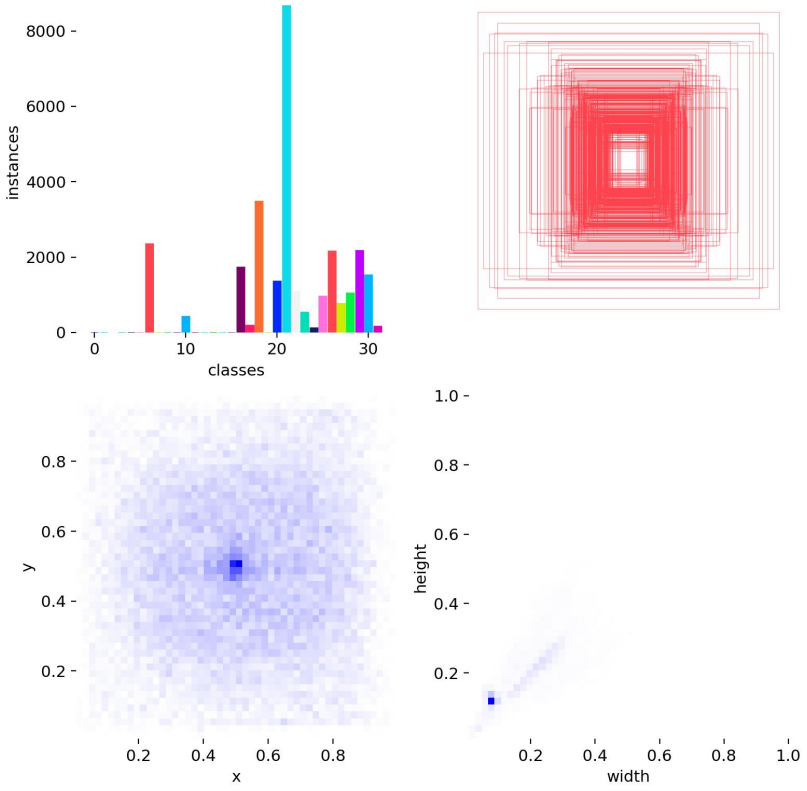

Figure 8.3: Labels

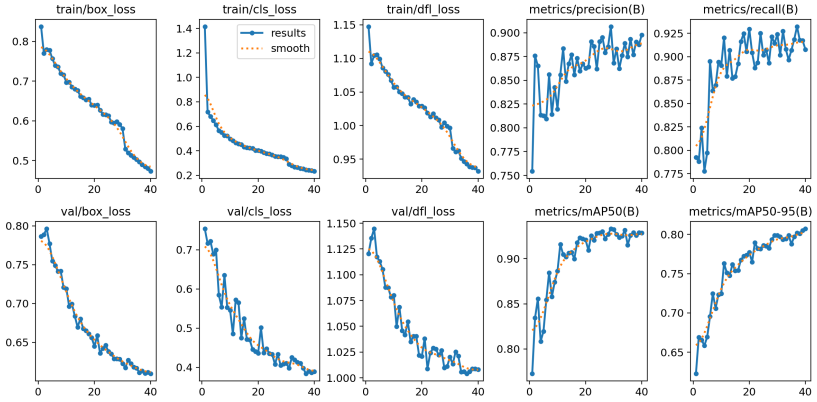

Figure 8.4: Summary Results

Chapter 9

Output and System Demonstration

9.1 Live Detection with Bounding Box Visualization

The system captures images from the ESP32-CAM and processes them on the laptop in real-time using the YOLOv8m model. Detected fruits are displayed immediately with bounding boxes, class labels, and confidence scores overlaid on each frame. This combined output demonstrates the system's ability to perform continuous, accurate detection of multiple fruit classes under varying lighting and orientations. The annotated frames provide clear visual feedback, allowing verification of predictions and insight into both correct detections and minor misclassifications, highlighting the practical effectiveness of the IoT-based fruit detection pipeline.

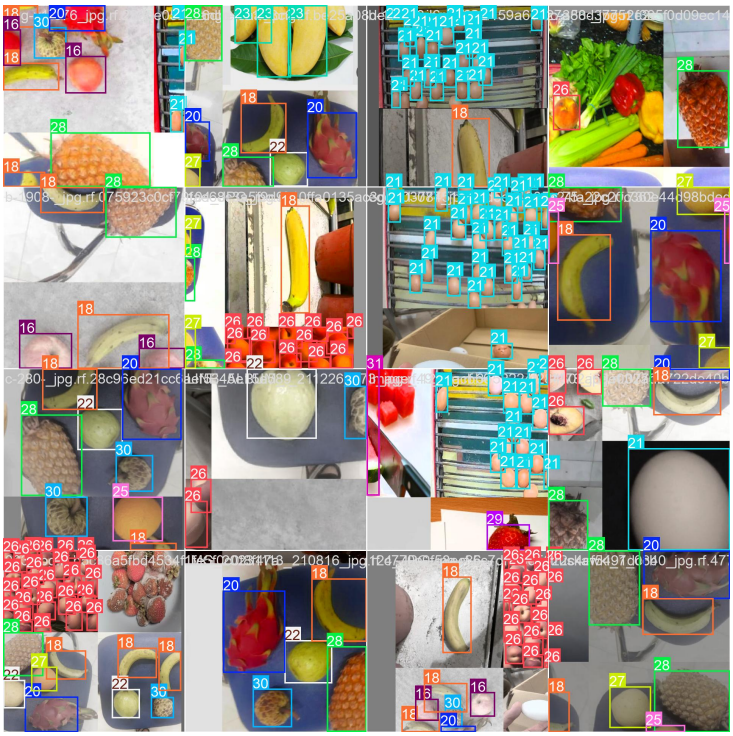

Figure 9.1: Bounding box visualization example 1

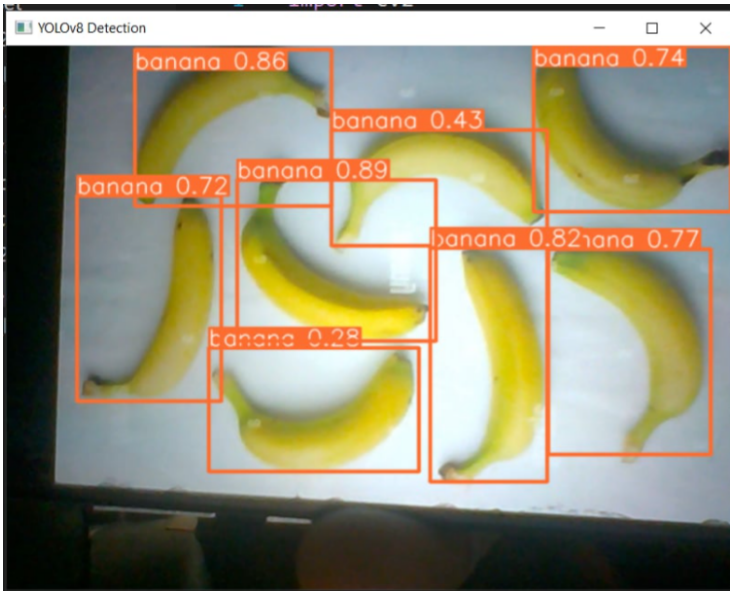

Figure 9.2: Bounding box visualization example 2

Chapter 10

Challenges and Limitations

10.1 ESP32-CAM Memory and Processing Constraints

- The ESP32-CAM has very limited RAM and processing capacity, restricting it to basic tasks such as image capture and transmission.
- It cannot run computationally heavy models like YOLO, making external processing on the laptop necessary.
- Frame buffer size is small, which limits resolution and can occasionally lead to dropped or corrupted frames under heavy load.
- Power consumption is also a concern, as long-term use on a power bank can reduce stability.

10.2 Network Stability and Bandwidth Issues

- The system depends on a mobile hotspot, which can suffer from fluctuating signal strength and interference.
- High-resolution frames require more bandwidth, slowing transfer rates and sometimes causing frame delays.
- Network instability can result in interruptions, reducing the smoothness of live detection output.
- The reliance on Wi-Fi means performance drops sharply in areas with weak signals.

10.3 Python Inference Bottlenecks

On the laptop side, the Python script handles frame fetching, decoding, inference, annotation, and display sequentially. This sequential processing introduces delays, especially when working with higher frame rates. The use of YOLOv8m, while accurate, adds additional computation time compared to smaller variants such as YOLOv8n. As a result, the system faces a trade-off between accuracy and real-time responsiveness. Without further optimization or hardware acceleration (e.g., GPU inference), the frame rate may remain below ideal levels for fully smooth video output.

Chapter 11

Conclusion

This project successfully demonstrated an IoT-based fruit detection system using an ESP32-CAM module and a YOLOv8m model trained on a custom dataset. By integrating the ESP32-CAM with a laptop through a shared wireless network, frames were captured and processed in real time, enabling accurate fruit recognition and visualization through bounding boxes. The implementation highlighted the potential of combining low-cost embedded devices with advanced deep learning models to deliver practical, real-world applications.

Despite these achievements, the project faced certain limitations. The ESP32-CAM's restricted memory and processing capacity required offloading inference tasks to the laptop, while network delays and Python-based processing created performance bottlenecks. These challenges emphasize the trade-off between hardware constraints and real-time system requirements in IoT-based image processing.

Looking ahead, future improvements can focus on optimizing the detection pipeline by deploying lightweight models (e.g., YOLOv8n or quantized versions) directly on the ESP32, enhancing real-time responsiveness. Cloud-based integration may also be explored for scalable processing and remote accessibility. Furthermore, extending the system to include real-time alerts, mobile application interfaces, or multi-camera support could make it more robust and adaptable for broader use cases such as smart agriculture and automated retail.

In summary, this work demonstrates a solid foundation for IoT-enabled computer vision, while leaving ample scope for further research and optimization.

Bibliography

[1] Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Mark Liao. Yolov4: Optimal speed and accuracy of object detection. *arXiv preprint arXiv:2004.10934*, 2020.

[2] CALORIES. fruits and veg 2 dataset. `https://universe.roboflow.com/calories/fruits-and-veg-2-x027v`, mar 2024. visited on 2025-09-14.

[3] Espressif Systems. *ESP32-CAM AI-Thinker Board Datasheet*. Espressif Documentation, 2022. Accessed: 2025-09-14.

[4] H. Huang, S. Yang, and Y. Zhang. Improved small-object detection using yolov8. In *Proceedings of the International Conference on Computer Vision Applications*, 2023. Accessed: 2025-09-14.

[5] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, 2016.

[6] Roboflow. How to train yolov8 object detection on a custom dataset. `https://blog.roboflow.com/how-to-train-yolov8-on-a-custom-dataset`, May 2023. Accessed: 2025-09-14.

[7] Roboflow. What is yolov8? a complete guide. `https://blog.roboflow.com/what-is-yolov8`, October 2024. Accessed: 2025-09-14.

[8] R. Santos and S. Rui. Esp32-cam video streaming web server. `https://randomnerdtutorials.com/esp32-cam-video-streaming-web-server-camera-home-assistant`. Accessed: 2025-09-14.

Chapter 12

Appendices

Appendix A: Full ESP32-CAM Code

Listing 12.1: ESP32-CAM Firmware Code

```
#include <WebServer.h>
#include <WiFi.h>
#include <esp32cam.h>

const char* WIFI_SSID = "TP-Link_8197";
const char* WIFI_PASS = "64633787";


WebServer server(80);


static auto loRes = esp32cam::Resolution::find(320, 240);
static auto midRes = esp32cam::Resolution::find(350, 530);
static auto hiRes = esp32cam::Resolution::find(800, 600);
void serveJpg()
{
  auto frame = esp32cam::capture();
  if (frame == nullptr) {
    Serial.println("CAPTURE-FAIL");
    server.send(503, "", "");
    return;
  }
  Serial.printf("CAPTURE-OK-%dx%d-%db\n", frame->getWidth(), frame->ge
                static_cast<int>(frame->size()));
```

```
  server.setContentLength(frame->size());
  server.send(200, "image/jpeg");
  WiFiClient client = server.client();
  frame->writeTo(client);
}

void handleJpgLo()
{
  if (!esp32cam::Camera.changeResolution(loRes)) {
    Serial.println("SET-LO-RES-FAIL");
  }
  serveJpg();
}

void handleJpgHi()
{
  if (!esp32cam::Camera.changeResolution(hiRes)) {
    Serial.println("SET-HI-RES-FAIL");
  }
  serveJpg();
}

void handleJpgMid()
{
  if (!esp32cam::Camera.changeResolution(midRes)) {
    Serial.println("SET-MID-RES-FAIL");
  }
  serveJpg();
}


void  setup(){
  Serial.begin(115200);
  Serial.println();
  {
    using namespace esp32cam;
    Config cfg;
    cfg.setPins(pins::AiThinker);
    cfg.setResolution(hiRes);
    cfg.setBufferCount(2);
```

```
    cfg.setJpeg(80);

    bool ok = Camera.begin(cfg);
    Serial.println(ok ? "CAMERA-OK" : "CAMERA-FAIL");
  }
  WiFi.persistent(false);
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
  }
  Serial.print("http://");
  Serial.println(WiFi.localIP());
  Serial.println("--/cam-lo.jpg");
  Serial.println("--/cam-hi.jpg");
  Serial.println("--/cam-mid.jpg");

  server.on("/cam-lo.jpg", handleJpgLo);
  server.on("/cam-hi.jpg", handleJpgHi);
  server.on("/cam-mid.jpg", handleJpgMid);

  server.begin();
}

void loop()
{
  server.handleClient();
}
```

Appendix B: Full Python Detection Script

Listing 12.2: Python YOLOv8 Detection Script

```
import cv2
import numpy as np
import requests
import os
import time
from ultralytics import YOLO

FRUITS = {"banana", "apple", "orange", "mango", "guava","strawberry","
```

```
# === CONFIG ===
url = 'http://192.168.1.102/cam-mid.jpg'  # ESP32-CAM snapshot URL
model_path = 'best.pt'

# === FILE CHECK ===
if not os.path.isfile(model_path):
    print(f"File not found: {model_path}")
    exit()

# === LOAD MODEL ===
model = YOLO(model_path)

# === MAIN LOOP ===
while True:
    try:
        # Get snapshot from ESP32-CAM
        response = requests.get(url, timeout=5, stream=True)
        if response.status_code != 200 or not response.content:
            print("Failed to fetch image. Skipping frame.")
            continue

        data = np.asarray(bytearray(response.content), dtype=np.uint8)
        im = cv2.imdecode(data, cv2.IMREAD_COLOR)

        if im is None or im.shape[0] == 0 or im.shape[1] == 0:
            print("Failed to decode image. Skipping.")
            continue

        # Flip image (rotate 180°)
        im = cv2.flip(im, -1)

        # YOLOv8 inference
        results = model.predict(im, conf=0.25)
        annotated = im.copy()

        for r in results:
            for box in r.boxes:
                cls_id = int(box.cls[0])
                conf = float(box.conf[0])
                label = model.names[cls_id].lower()
```

```
                    x1, y1, x2, y2 = map(int, box.xyxy[0])
                    cv2.rectangle(annotated, (x1, y1), (x2, y2), (255, 0,
                    cv2.putText(
                        annotated,
                        f"{label.upper()} {int(conf*100)}%",
                        (x1, y1 - 10),
                        cv2.FONT_HERSHEY_SIMPLEX,
                        0.6,
                        (255, 0, 255),
                        2
                    )



                # Show annotated frame
                cv2.imshow("YOLOv8 ESP32-CAM", annotated)

                if cv2.waitKey(1) & 0xFF == ord('q'):
                    break

                # Optional small delay
                time.sleep(0.05)

            except Exception as e:
                print(f"Error: {e}")
                continue

cv2.destroyAllWindows()
```